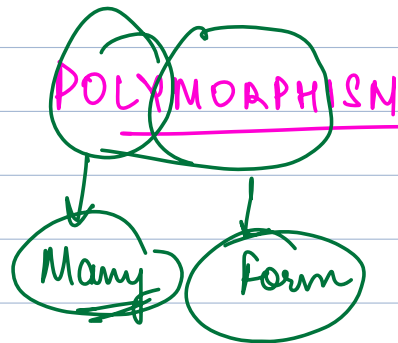
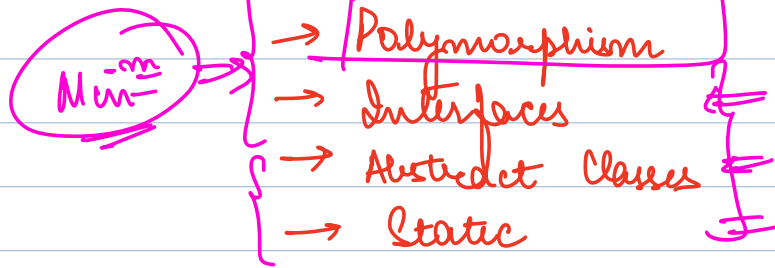


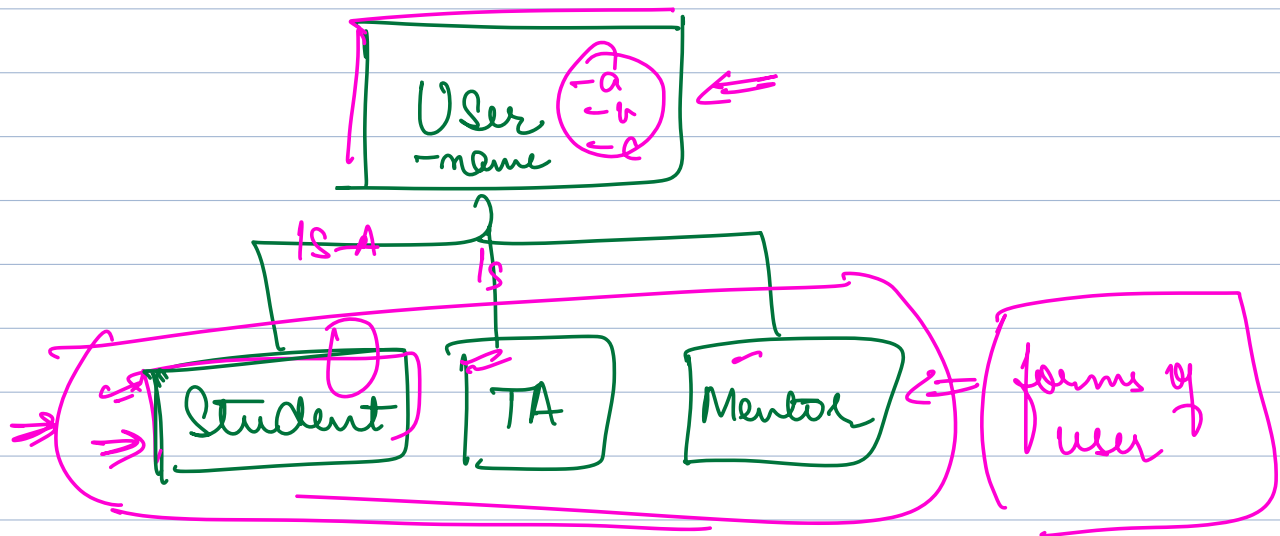
Remaining OOP

Today

Will start
at 9:10PM

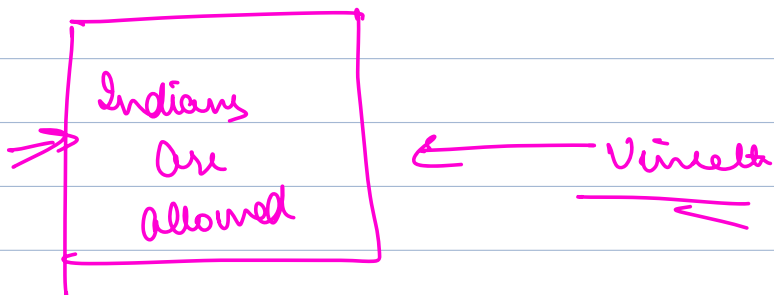
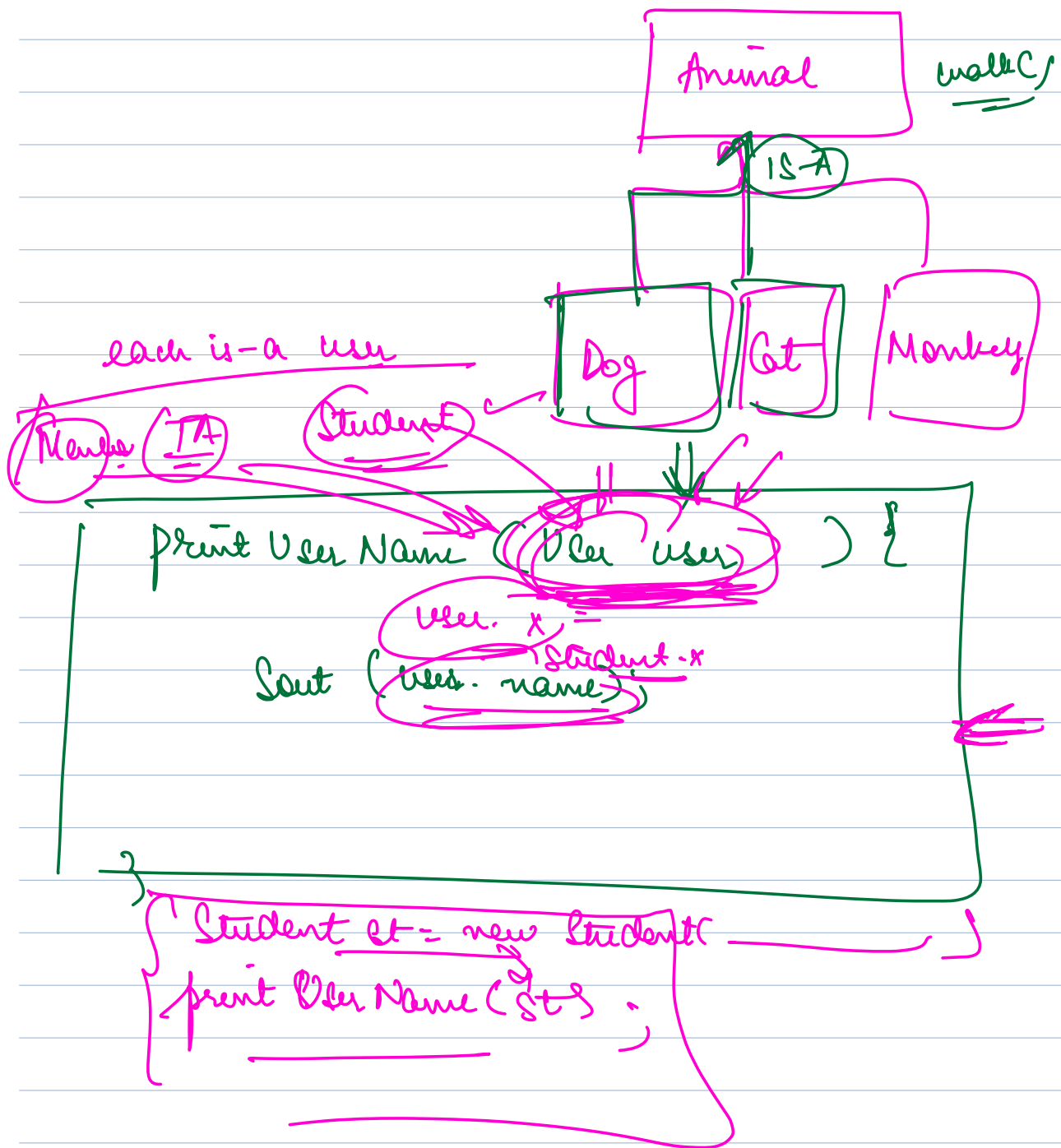


used to rep something that has multiple forms.



User has multiple forms.

- Student IS-A user
- ta IS-A user
- mentor IS-A user
- instructor IS-A user



```

    }
    get User() {
        return new Student();
        new TAC;
    }

```

```

list <User> users {
    new User(),
    new TAC,
    new Mentor(),
    new Student(),
}

```

```

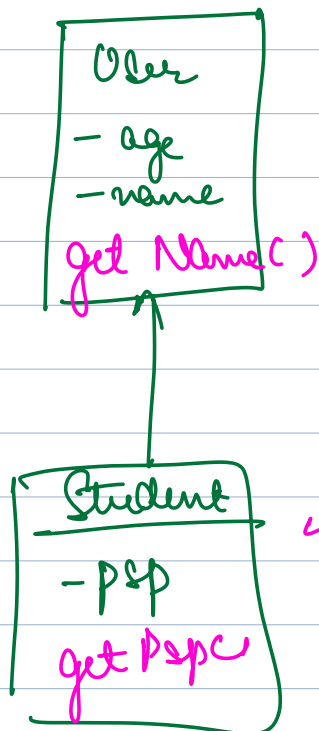
}
Restaurant {
    list <Indian> indians {
        virat,
    }
}

```

User v = new Student();

Anywhere where data type is of a parent class (User), you can put objects of any child class of that as well like.

User v = new Student() ^{get User()}



cout (v. name) ✓
cout (v. age) ✓
cout (v. pdp)

~~User v = get User();~~ ^{User is class, TA, Member}
~~v. pdp;~~
⇒ Not even compile

Computer only allows access to properties based on data

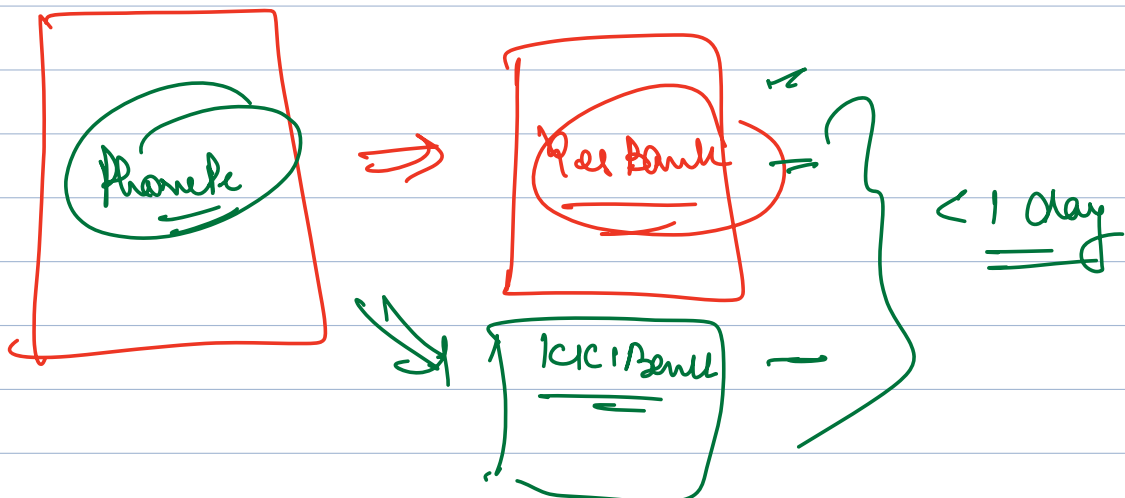
{ type of variable, and not
the real obj. }

```
get User() {  
    int i = rand() % 2;  
    if var i == 1:  
        ==  
    }  
    else i == 2  
        - -  
}
```

User u = new Student();
V/S Student s = new Student();

first is always a better approach

→ The more general your code is, the easier it is to extend.



Prompt

Yes Bank yb =

Testing

Bank

Yes Bank

~~ICICI Bank~~

~~Bank~~

~~(b)~~

~~new YesBank()~~

new ICICIBank()

Change Password { (User o) }

→ u. let Password (new Password)

?

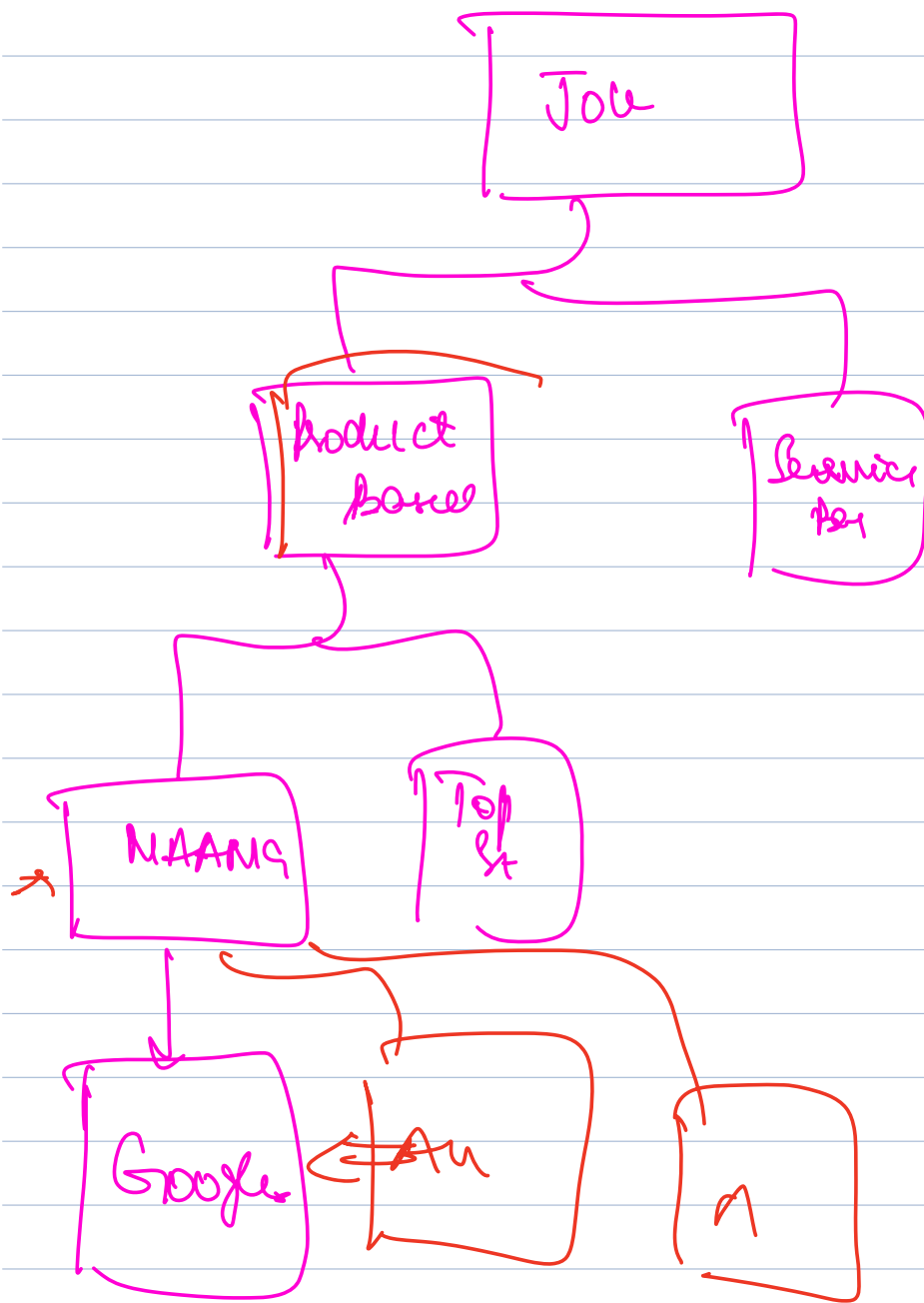
Change Password (new Student)

{ update Batch (Student send) {
?
}

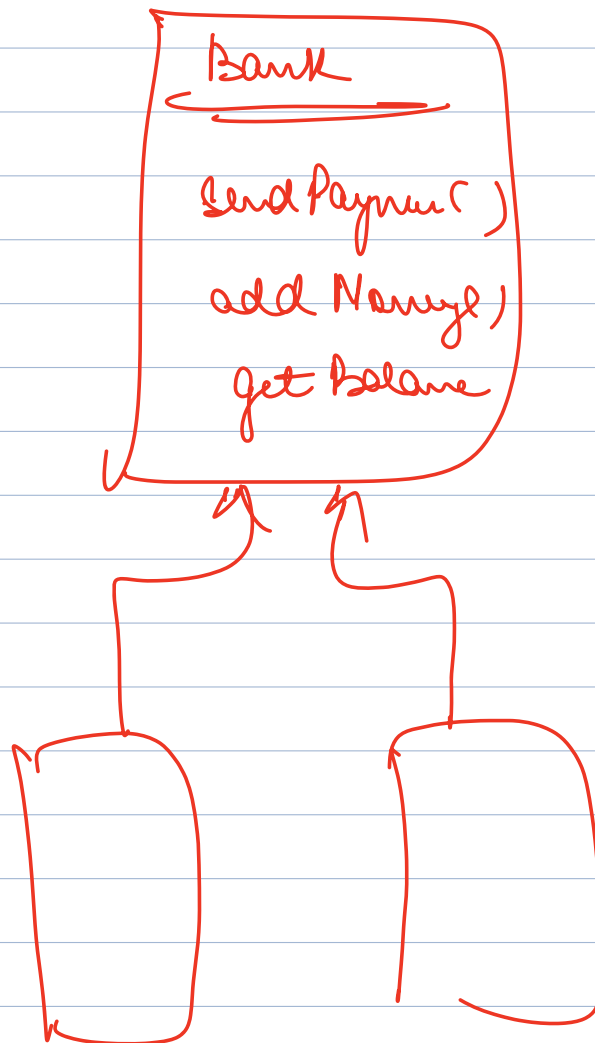
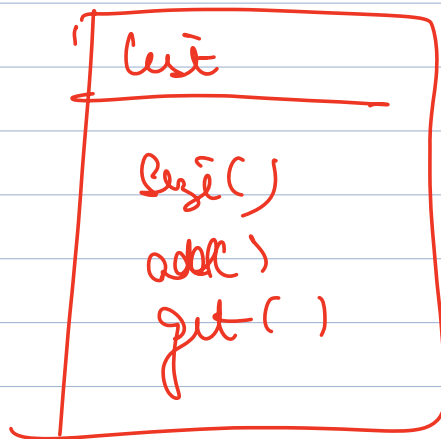
→ It is possible to put obj of child class in variable of parent class.

→ But you will be able to access only attr that are defined in parent

→ User Student
[list < User > = new ArrayList < >]



Create data type of variable to be as general class as possible.



Polymorphism

(1) Inheritance

(2) Method Overloading

↳ Compile Time Polymorphism

METHOD OVERLOADING

class A {

void hello() {

 cout << "Hello, World";

}

void hello (String name) {
 cout << "Hello, " << name << " ";
}

⇒ hello method has 2 forms
 (1) that takes no param
 (2) takes 1 param

A a = new A();

a. hello() → Hello World

a. hello("Naman") → Hello Naman.

Method Overloading : multiple methods
in a class with same
name but diff Signature

method
name (data types)

→ ① hello()
→ ② hello (String)

Method Overloading

→ 2 methods with same signature can't
be there in a class.
can't

A {
 void hello (int age) ;
 String hello (int age) ;
}

error

⇒ hello (int)
⇒ hello (int)

```

A {
    void hello(int age);
    String hello(int pep);
}

```

↓

hello(int)

hello(int)

~~hello(int age, int rank)~~
~~hello(int rank, int age)~~

```

Client {
    int hello(String)
    String hello(String)
}

```

hello("Naman")

hello(String, int)

hello(int, String)

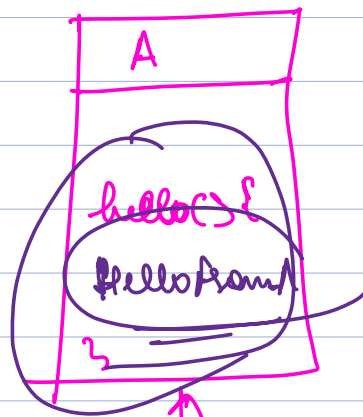
hello (User)
↑
hello (Student)

POLYMORPHISM

- ① Inheritance
- ② Method Overloading
- ③ Method Overriding

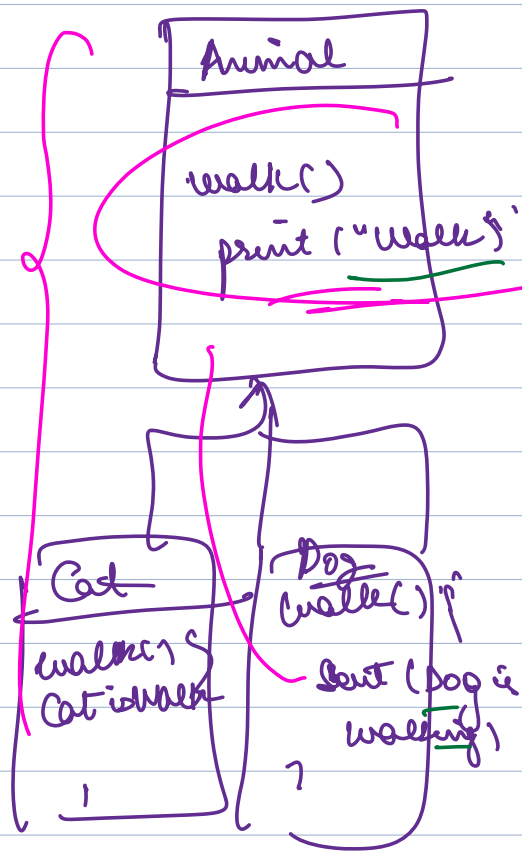
Method Overriding

→ replace / update / change / bypass



```
B b = new B();  
b.hello();
```

- ⇒ child classes must have all beh of their parents
- ⇒ But child classes may want to perform that behaviour in a diff way

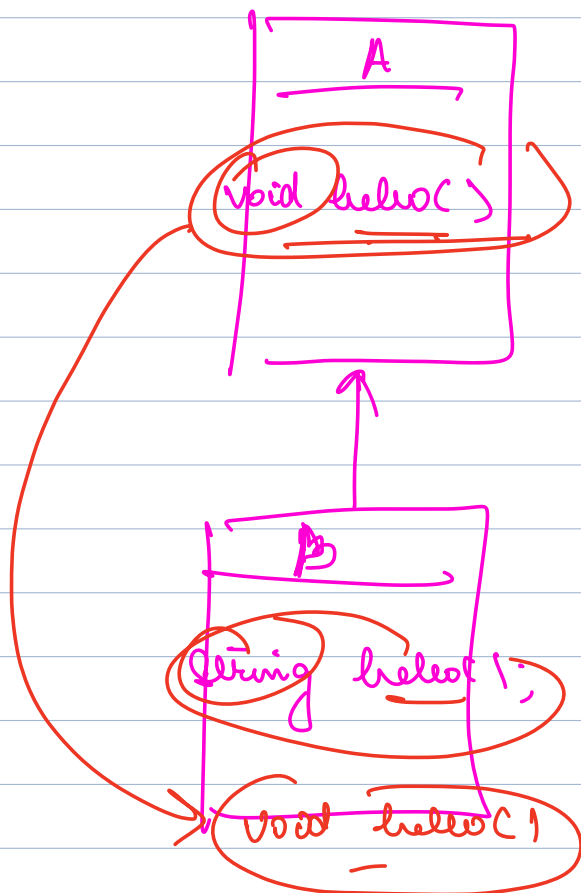
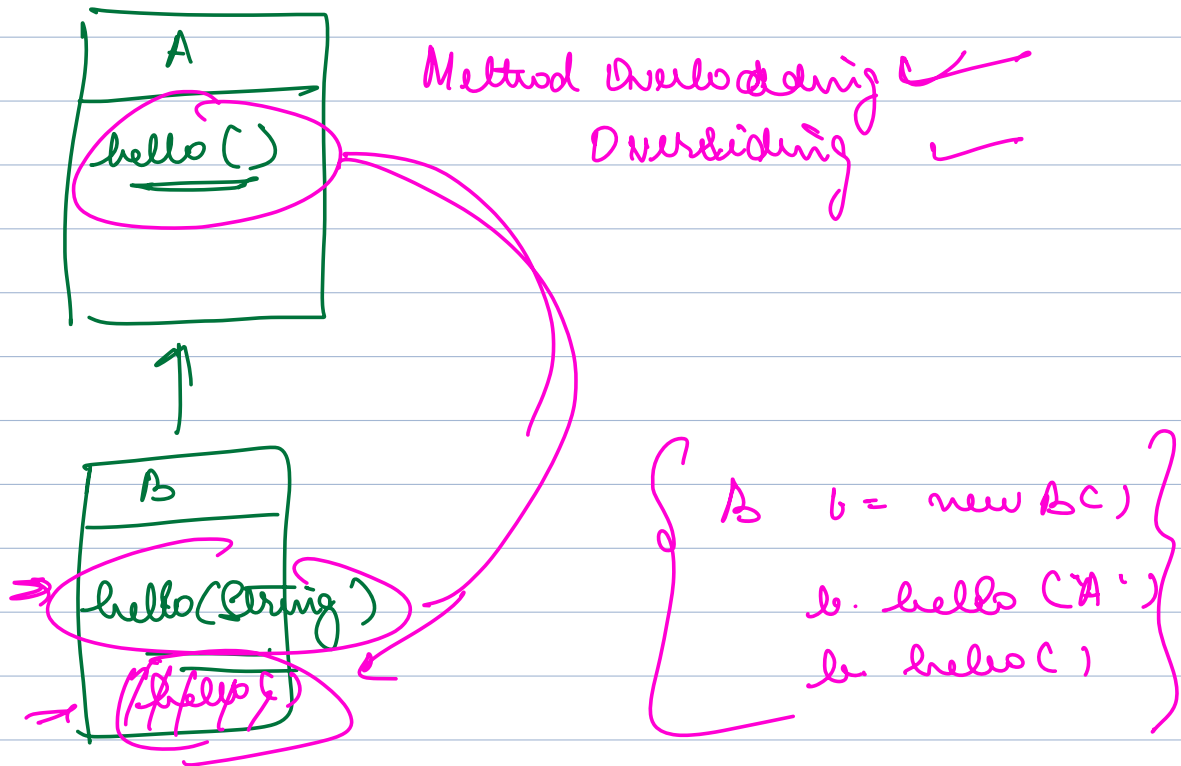


⇒ When a child class has a method which is exactly same as parent
 (Same signature + same return type)
 the child's method replaces the parent's method

```

Dog d = new Dog()
d.walk()
  
```

Method Overriding



Overriding X

Overloading X

Compile Time Error ✓

```
Animal a = new Dog()
a.walk()
```

- ① Walk ~~✗~~
- ② Dog is walking. ✓

```
Animal a = new Dog()
a.walk()
```

→ Dog is walking

⇒ Method == Code ⇒ Executed at runtime ⇒ Code / method of real obj (not data type) will be called.

- ① The method that is executed is always of the obj that is actually present in the variable.
- ② At the compile time, compiler observes the behaviour of variable

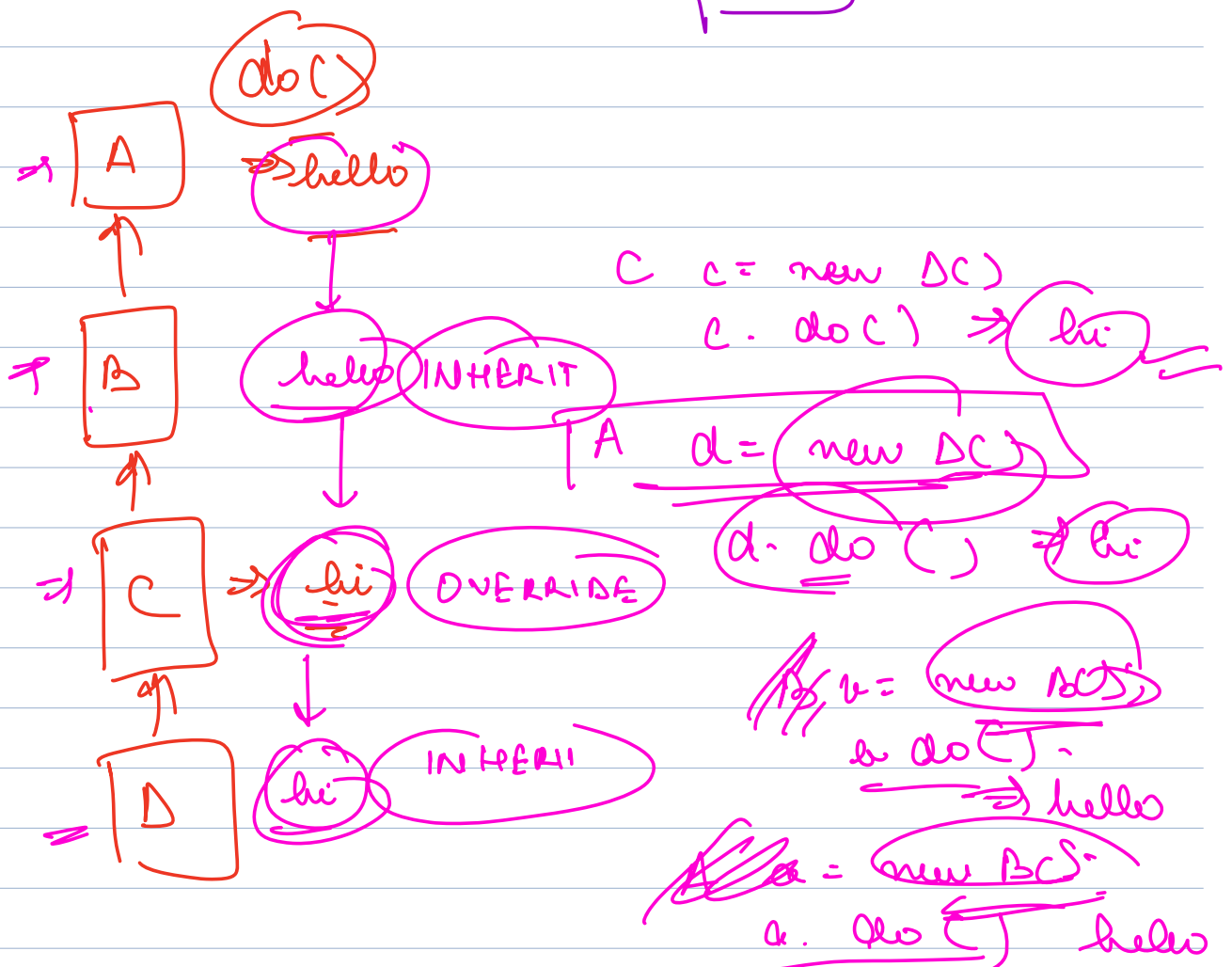
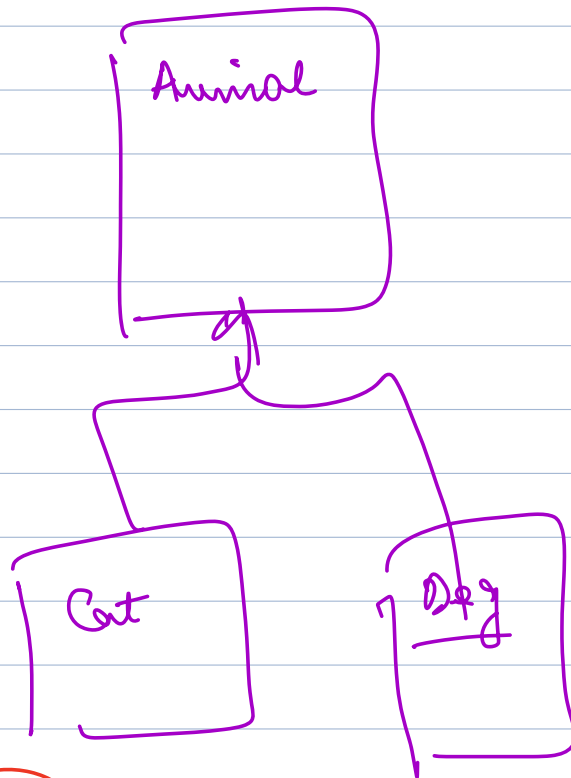
COMPILER $\Rightarrow$ data type of variable

RUNTIME $\Rightarrow$ data type of obj

A { doC() { <u>hello</u> } }	B extends A { doC() { <u>Bye</u> } }	C extends A { doC() { Hi } }
	D extends A {	

```
List<A> as { new A(), new B(), new C(), new D() }  
for (A a : as)  
    a.doC()
```

hello Bye Hi hello



B/ b = new CC()
b. do() → hi

→ after class, just try out random case

Δ a = new AC()
~~X IS NOT A~~
Student s = new User()
s. pp
User u = new Student()
IS-A

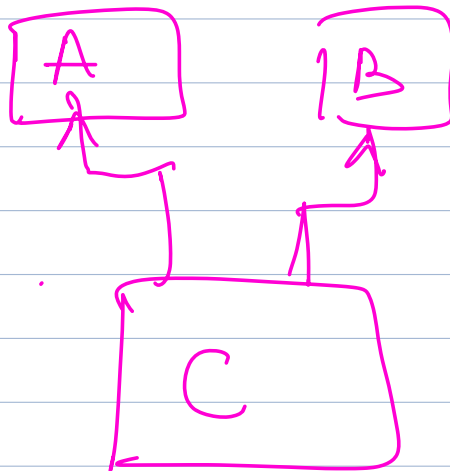
User u = (new Student());
u. updateBatch();

(Student)user, update
Batch (1)

list l = new ArrayList<>()

User u = new Student

list.add(u) user



~~C extends A, B~~

C++ ++C

++C++ ++C ++C